Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Прикладная математика

Отчет по лабораторной работе №1

Работу выполнили:

Привезенцев Андрей

Шароватов Вадим

Сиомов Дмитрий

Факультет: ИТИП

Группа: M32061

Преподаватель:

Свинцов Михаил Викторович

Санкт-Петербург 2022

Методы нахождения производной

Теория

Методы преимущественно отличаются количеством узлов, задействованных в вычислении, а также их расположением относительно точки, в которой находится производная.

В данной лабораторной работе рассматриваются три метода. Два из них имеют первый порядок точности:

1. Правая разностная производная
$$f'(x) \cong \frac{f(x+h) - f(x)}{h}$$
2. Левая разностная производная
$$f'(x) \cong \frac{f(x) - f(x-h)}{h}$$

И метод центральной разностной производной, имеющий второй порядок:

$$y_i' = \frac{y_{i+1} - y_{i-1}}{2h}$$
$$y_1' = \frac{-3y_0 + 4y_1 - y_2}{2h}$$
$$y_n' = \frac{y_{n-2} + 4y_{n-1} - 3y_n}{2h}$$

Первые два метода отличаются друг от друга только расположением узлов относительно искомого, а в третьем методе используются три узла.

Реализация (Задание №1)

Три метода нахождения производной:

```
#Правая разностная производная
def forward_difference(func: callable, x0, step: float):
    return (func(x0 + step) - func(x0)) / step

#Центральная разностная производная
def central_difference(func: callable, x0, step: float):
    return (func(x0 + step) - func(x0 - step)) / (2 * step)

#Левая разностная производная
def backward_difference(func: callable, x0, step: float):
    return (func(x0) - func(x0 - step)) / step
```

Результат работы программы при f(x) = sin(x), x_0 = 1 и h = 0.1:

Правая разностная производная: 0.4973637525353891

Центральная разностная производная: 0.53940225216976

Левая разностная производная: 0.5814407518041309

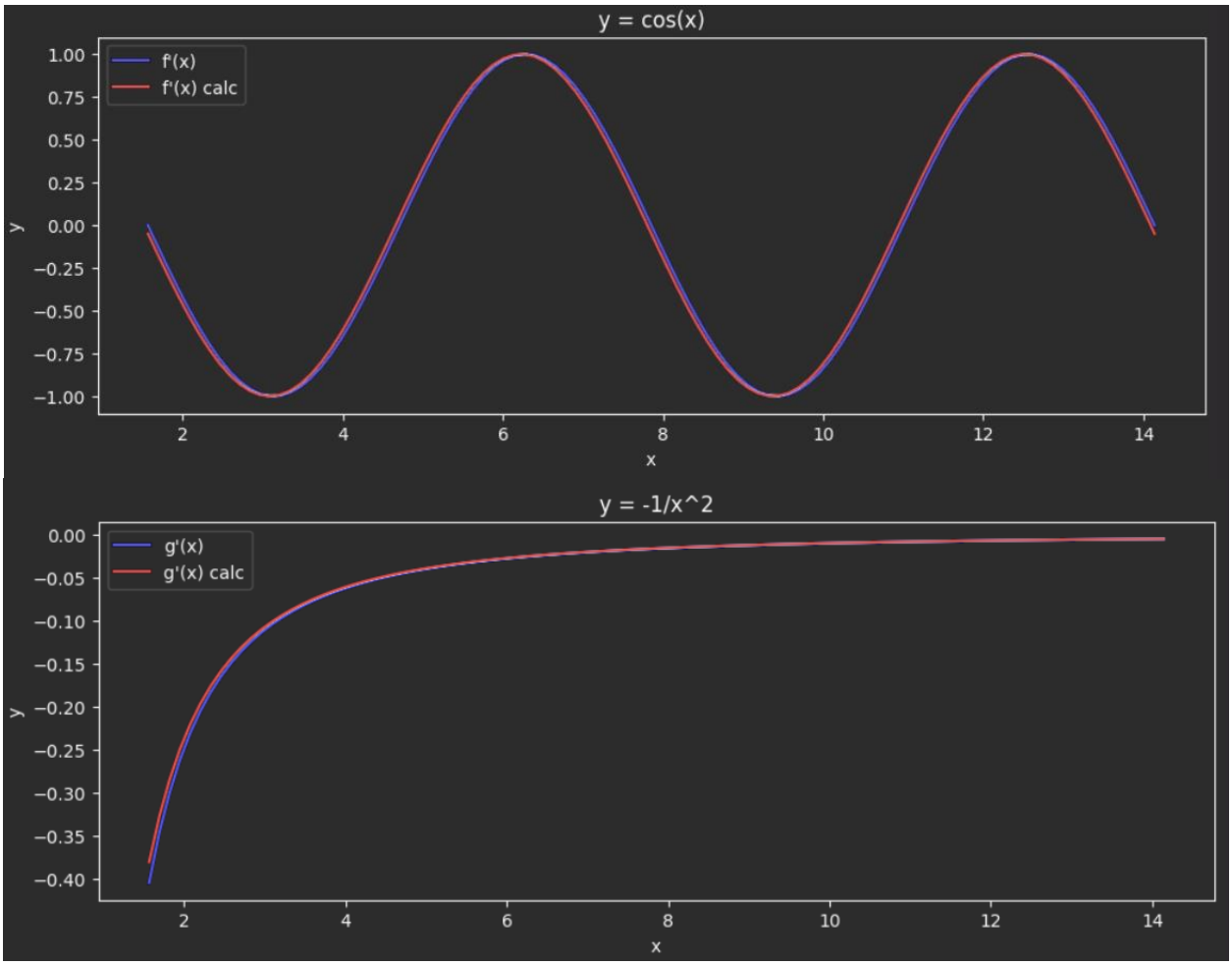
Задание №2

Вычислим численные значения производных для функций f(x) = sin(x) и g(x) = 1/x

Построим графики и сравним их с графиками функций соответствующих производным данных функций.

$$\frac{df}{dx} = \cos(x) \quad \frac{dg}{dx} = -\frac{1}{x^2}$$

Для большей наглядности воспользуемся методом правой разностной производной т. к. при использовании более точного метода средней разностной производной, графики накладываются друг на друга и сливаются. h = 0.1



Задание №3

Среднеквадратичное отклонение – показатель рассеивания значений случайной величины относительно её математического ожидания

$$\sigma = \sqrt{\frac{\sum(x_i - \bar{x})^2}{n-1}}$$

В нашем случае рассматривается отклонение значений, полученных численно от значений, полученных аналитически.

Пример программы для метода правой разностной производной при f(x) = sin(x)

```
x3 = np.linspace(0, np.pi, 100)

true_derivatives = f_prime(x3)

dx = x3[1] - x3[0]
numerical_derivatives = forward_difference(f, x3, dx)

std_dev = np.std(true_derivatives - numerical_derivatives)

print("Standard deviation: ", std_dev)
```

Результат работы программы:

Правая разностная производная: 0.004964180655537803

Центральная разностная производная: 0.0001192619560685242

Левая разностная производная: 0.004964180655536632

Вывод:

Метод центральной разностной производной оправдывает свой более высокий порядок точности, в то время как два других метода имеют очень близкие отклонения что не удивительно

Задание №4

Рассмотрим зависимость среднеквадратичного отклонения от шага метода. Напомним, что шаг метода это – h

Нахождение множества отклонений метода правой разностной производной на примере f(x) = sin(x):

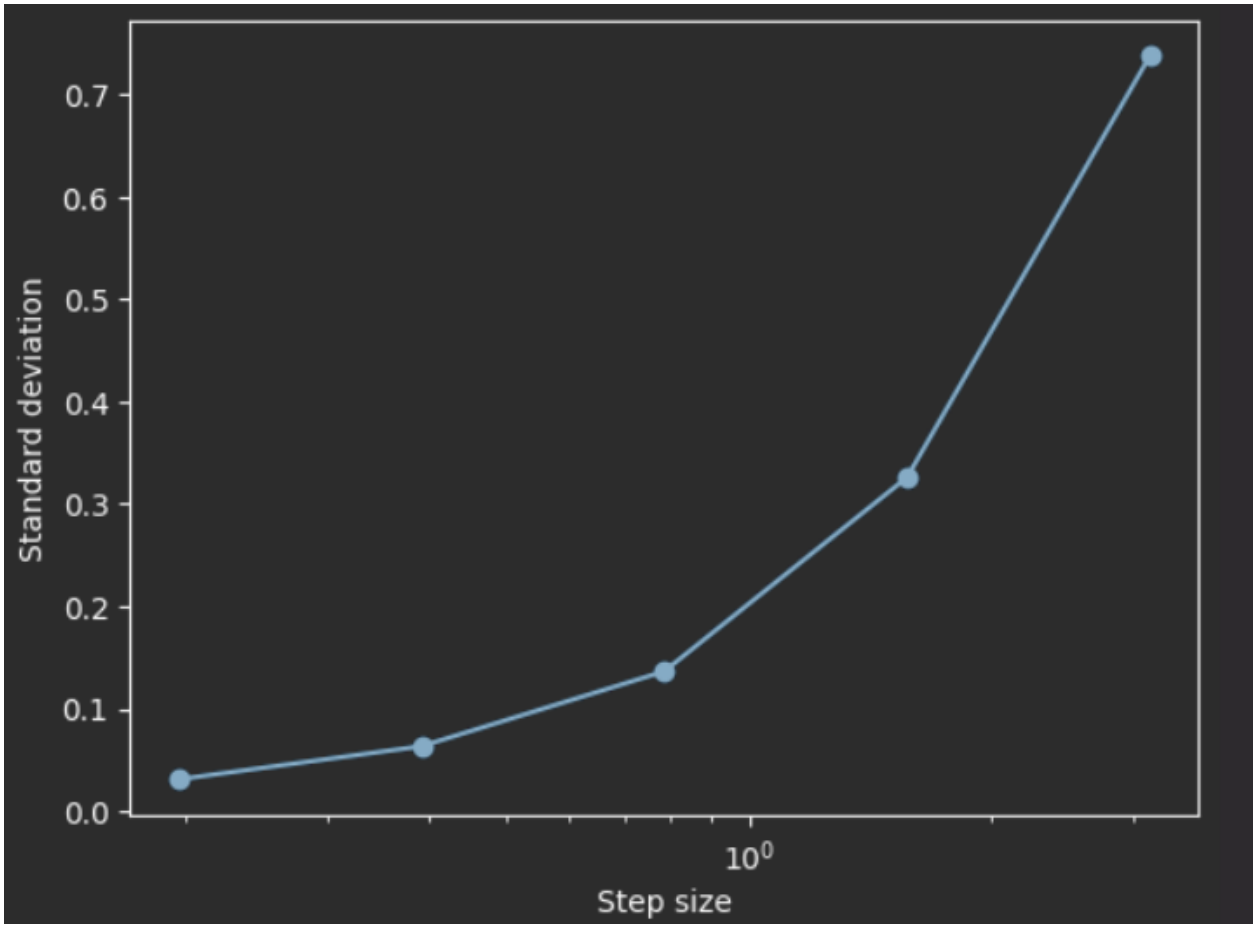
```
x4 = np.linspace(0, np.pi, 100)

true_derivatives = f_prime(x4)

steps = [np.pi / 2 ** i for i in range(5)]

std_devs = []
for step in steps:
    numerical_derivatives = forward_difference(f, x4, step)
    std_dev = np.std(true_derivatives - numerical_derivatives)
    std_devs.append(std_dev)
```

Построим график полученного множества, с помощью которого определим зависимость:



По графику видно, что при увеличении шага среднеквадратичное отклонение растет (логично)

Методы численного интегрирования

Теория

Для нахождения приближенного значения определенного интеграла могут использоваться так называемые квадратурные формулы

$$I = \int_a^b f(x)dx \cong \sum A_i f(\bar{x}_i)$$

Методы численного интегрирования отличаются способом вычисления площадей элементарных трапеций, на которые разбивается вся криволинейная трапеция интеграла.

В данной лабораторной рассматриваются три(5) метода:

1. Формула прямоугольников (3)

$$I_i \cong hf_{i-1} \qquad I_i \cong hf_i \qquad I_i \cong hf_{i+1}$$

2. Формула трапеций

$$I_i \cong \frac{h}{2}(f_{i-1} + f_i)$$

3. Формула Симпсона

$$I_i \cong \frac{h}{6}(f_{i-1} + 4f_{i-\frac{1}{2}} + f_i)$$

Реализация (Задание №5)

Методы нахождения интеграла:

```
def left_rectangles(func: callable, left: float, right: float, step: float) -
> float:
    result = 0
    i = 1
    while left + step * i <= right:
        result += step * func(left + step * (i - 1))
        i += 1

    return result


def right_rectangles(func: callable, left: float, right: float, step: float)
-> float:
    result = 0
    i = 1
    while left + step * i <= right:
        result += step * func(left + step * i)
        i += 1

    return result


def central_rectangles(func: callable, left: float, right: float, step:
float) -> float:
    result = 0
    i = 1
    while left + step * i <= right:
        result += step * func(left + step * (i - 0.5))
        i += 1

    return result


def trapezoid(func: callable, left: float, right: float, step: float) ->
float:
    result = 0
    i = 1
    while left + step * i <= right:
        result += 0.5 * step * (func(left + step * (i - 1)) + func(left +
step * i))
        i += 1

    return result


def simpson(func: callable, left: float, right: float, step: float) -> float:
    result = 0
    i = 1
    while left + step * i <= right:
        result += 1/6 * step * (func(left + step * (i - 1)) + 4 * func(left +
step * (i - 0.5)) + func(left + step * i))
        i += 1

    return result
```

Задание №6

Вычислим интегралы для функций f(x) = sin(x) и g(x) = 1/x и сравним их со значениями найденными привычным путем

Результаты:

f(x) = sin(x)

Теоретические значения: 0.0

Метод левых прямоугольников: -0.01574983863265133

Метод правых прямоугольников: -4.649058915617843e-16

Метод центральных прямоугольников: -0.007890489394735554

Метод трапеций: -0.007874919316325106

Метод Симпсона: -0.007885299368598268

g(x) = 1/x

Теоретические значения: 1.6094379124341003

Метод левых прямоугольников: 1.617501375022094

Метод правых прямоугольников: 1.5855658911511261

Метод центральных прямоугольников: 1.601341802541127

Метод трапеций: 1.6015336330866097

Метод Симпсона: 1.6014057460562874

Задание №7

Рассмотрим зависимость среднеквадратичного отклонения от шага метода. Для этого найдем среднеквадратичное отклонение для различных шагов и построим график

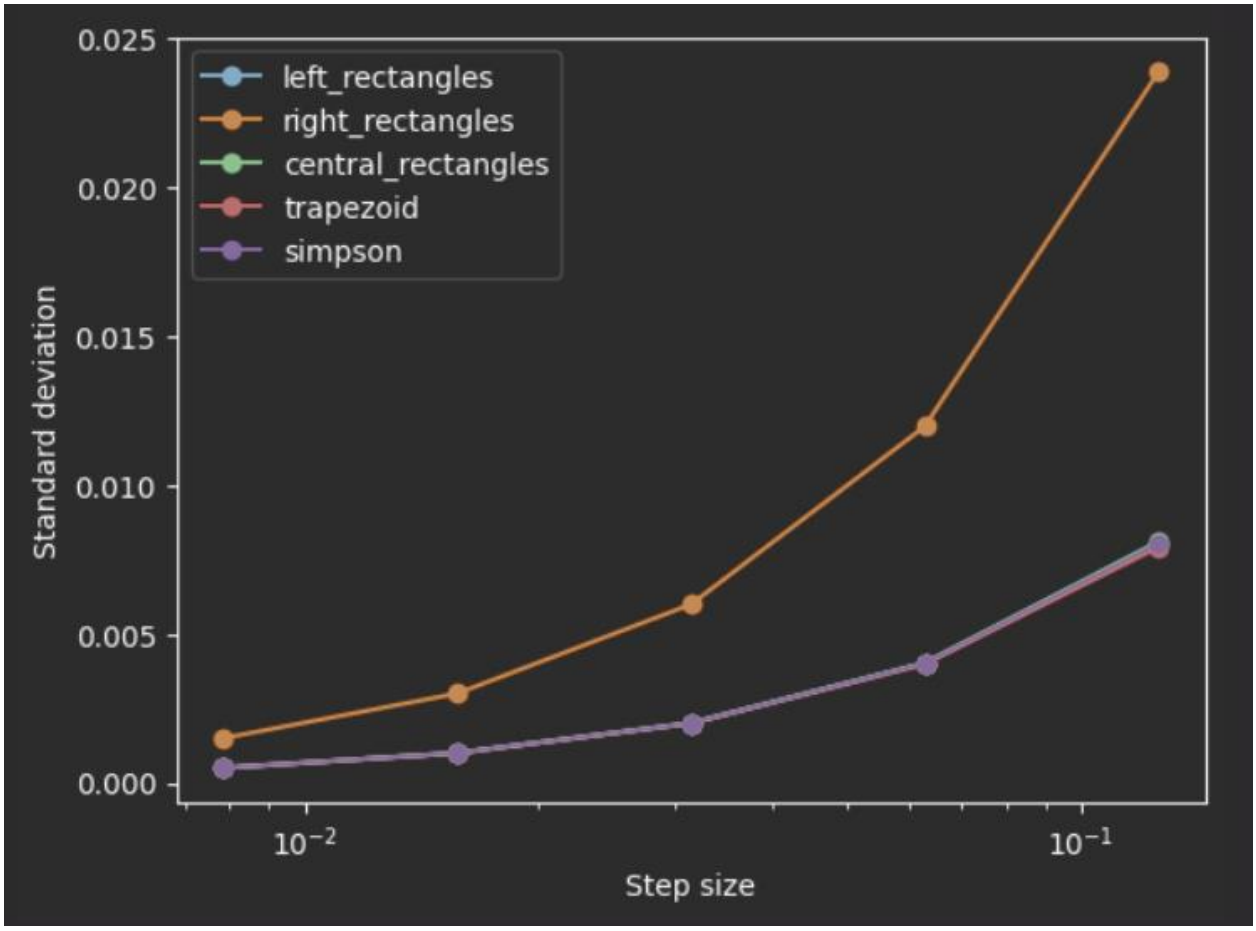
Пример программы для функции g(x) = 1/x:

```
steps = [dx / 2 ** i for i in range(5)]

std_devs = [[0] * len(steps) for i in range(5)]
G = calc_integral(g_integral, xL, xR)

for i, step in enumerate(steps):
    std_devs[0][i] = abs(left_rectangles(g, xL, xR, step) - G)
    std_devs[1][i] = abs(right_rectangles(g, xL, xR, step) - G)
    std_devs[2][i] = abs(central_rectangles(g, xL, xR, step) - G)
    std_devs[3][i] = abs(trapezoid(g, xL, xR, step) - G)
    std_devs[4][i] = abs(simpson(g, xL, xR, step) - G)
```

Результат работы программы:



Ситуация аналогичная.